

Sistemas distribuidos 2026

Proyecto 1

Integrantes: Schnaider Germán, Infante Gerónimo

Aclaración general para la correcta ejecución de cada inciso:

En cada carpeta se encuentra un archivo make, el cual permite compilar y crear los archivos ejecutables.

Por ello antes de realizar los comandos para ejecutar en consola, es necesario ejecutar el archivo make que se encuentra en el directorio del ejercicio, o en caso de estar dividido en cliente y servidor, ejecutar el make en cada carpeta.

1a- Para probar este ejercicio se tiene que hacer

- proyecto1/ejercicio1/a\$./padre_execv.out
- proyecto1/ejercicio1/a\$./padre_system.out

La salida de este es:

schnaider@German:~/universidad/sd/proyecto1/ejercicio1/a\$./padre

Hijo: imagen cargada, ejecutando trabajo...

Tiempo desde fork() hasta que el hijo terminó: 1215 microsegundos

schnaider@German:~/universidad/sd/proyecto1/ejercicio1/a\$./padre_system

Hijo: imagen cargada, ejecutando trabajo...

Tiempo desde system() hasta que el hijo terminó: 2968 microsegundos

Podemos ver que usando execv tenemos un menor tiempo de creación del hijo, ya que es más bajo nivel y tiene menos overhead.

1b- Para probar este ejercicio se tiene que hacer

- En máquina cliente:
proyecto1/ejercicio1/b/Cliente\$./clienteEjecutable.out <IP-servidor>
- En máquina servidor:
proyecto1/ejercicio1/b/Servidor\$./ejecutable_server_execv.out

y luego para siguiente ejecución:

- En máquina cliente:
proyecto1/ejercicio1/b/Cliente\$./clienteEjecutable.out <IP-servidor>
- En máquina servidor:
proyecto1/ejercicio1/b/Servidor\$./ejecutable_server_system.out

Los tiempos de salidas obtenidos a partir de ejecutar el Cliente en una máquina Ubuntu (WSL) y los servidores en RaspbianOS:

Tiempo hasta que el hijo terminó (Execv): 2206 microsegundos

Tiempo hasta que el hijo terminó (System): 4388 microsegundos

2- Para ejecutar este código se tiene que hacer "proyecto1/ejercicio2/server\$

./serverEjecutable.out" en la máquina que va a funcionar como servidor y

"proyecto1/ejercicio2/client\$./clienteEjecutable.out <ip servidor>" en la máquina servidor.

a- Tiempo hasta que el hijo terminó: 0.000731 segundos

- b- Tiempo hasta que el hijo terminó: 0.007914 segundos (Servidor en WSL, Cliente en WLS de otra máquina ambas con windows 11 como sistema operativo host)
- c- Tiempo hasta que el hijo terminó: 0.009251 segundos (Servidor en WSL, Cliente en RPI).

3- Para ejecutar este código se debe ejecutar en la máquina servidor

“proyecto1/ejercicio3/server\$./ejecutable_servidor.out” y ejecutar en la máquina cliente “proyecto1/ejercicio3/client\$./ejecutable_cliente.out <ip server>”. Estos son los resultados para una ejecución en la misma maquina y en diferentes maquinas (WSL , RPI), respectivamente:

Tiempo transcurrido: 0.000963588 segundos

Tiempo transcurrido: 0.004550103 segundos

4-Pasos a realizar para ejecutar:

Inciso A:Ejecutar: proyecto1/ejercicio4/a\$./ejecutable_programaMain.out

Inciso B: Ejecutar

-En máquina de servidor: proyecto1/ejercicio4/b/server\$./serverEjecutable.out

-En máquina cliente: proyecto1/ejercicio4/b/client\$./clienteEjecutable.out <IP-Servidor>

En la siguiente tabla se presentan los tiempos de ejecución obtenidos: en el Inciso A, los procesos se ejecutaron localmente sobre una máquina virtual WSL (Ubuntu); en el Inciso B, se utilizó comunicación remota entre dos máquinas físicas corriendo WSL con Ubuntu.

Inciso	Tiempo de ejecución
A (WSL Ubuntu)	728222 ns
B(WSL Ubuntu, Raspbian OS)	1246970 ns

Análisis de resultados:

Como se observa en los datos obtenidos, el tiempo de ejecución en el Inciso B presenta un orden de magnitud superior al del Inciso A. Esta diferencia resulta debido a la naturaleza de los mecanismos de comunicación utilizados: el Inciso A utiliza colas de mensajes para la comunicación interprocesos (IPC) dentro de un mismo host, mientras que en el Inciso B se utiliza Sun RPC para la comunicación entre procesos que se encuentran en distintos nodos.

Este incremento en la latencia es el comportamiento esperado, ya que la comunicación remota introduce el overhead propio de la transferencia de datos a través de la red, sumado al proceso de serialización y deserialización (marshalling/unmarshalling) de los mensajes realizado por los stubs.

5a- Para ejecutar se tiene que hacer “ejercicio5/consumer/ejecutable_consumer.out” en la maquina del consumidor, “ejercicio5/producer/ejecutable_producer.out” en la maquina del productor y “ejercicio5/queue_admin/ejecutable_queue_admin.out” en la maquina del queue admin que va a estar en el medio de la comunicación.

5b- Consideraciones de diseño: El sistema se modeló usando un patron tipo Intermediario, donde los productores y consumidores no se comunican de manera directa sino que se comunican a través de un servidor administrador.

- Protocolo de comunicación en sockets: Se eligió UDP para los sockets debido a que en otro ejercicio ya usamos TCP, además de que simplifica la implementación y manejo de varios clientes (productores y consumidores). Nos pareció correcto en el ambiente donde lo ejecutamos, ya que dentro de la misma LAN la tasa de errores es muy baja, además de que las tareas que producen/consumen son de un tamaño menor al limite de UDP.
- Hilos: En el Queue Admin, se usaron 2 hilos para escuchar en diferentes puertos a la vez y separar las responsabilidades del manejo de productores y consumidores. Para su sincronización usamos una cola de mensajes que es thread-safe.
- Elegimos que el servidor no empuje las tareas al consumidor a ciegas, sino que el consumidor tome la iniciativa y pida la tarea.

Consideraciones para la comunicación:

- Al usar UDP el servidor es stateless. Los clientes no necesitan hacer un handshake.
- Productores: El productor inicia su comunicación simplemente enviando un datagrama al puerto 8080. Al no requerir confirmación del servidor, el productor no se bloquea y puede seguir generando tareas a su propio ritmo. Para finalizar la comunicación puede irse sin problema debido a que el servidor es stateless.
- Consumidores: Como el servidor no mantiene conexiones abiertas, el servidor no conoce a los consumidores hasta que le envían algo. Por esto cuando un consumidor quiere consumir una tarea, envía un datagrama de un solo byte al puerto 8081 como trigger, enviando su <ip,puerto> dentro del mensaje. Con estos datos el Queue Admin puede darle una tarea cuando esté disponible. Para finalizar la comunicación puede irse sin problema debido a que el servidor es stateless, mientras que no sea en medio de la espera de la tarea que le entrega el Queue Admin.
- Queue Admin: Para iniciar la comunicación se tiene que ejecutar antes que los

6- El problema se planteó basándose en el algoritmo de Cristian, donde se midió el RTT de la conversación con cada servidor y se aproximó el tiempo de ida a $RTT/2$. Entonces se calculó la diferencia de hora teniendo en cuenta el tiempo de ida. Para ejecutarlo se hace “ejercicio6/master/ejecutable_master.out <ip servidor 1> <ip servidor 2> ...” en la máquina del proceso *cliente*, mientras que se hace “ejercicio6/slave/ejecutable_slave.out” en las máquinas de los servidores que se le va a consultar su hora.

7- Se podría sincronizar usando las diferencias de tiempos recibidos y utilizando el algoritmo de Berkeley, donde nuestro Master es el servidor de tiempo y los slave son los clientes del servidor de tiempo. De esta forma el servidor y todo cliente en la red se sincronizará (sincronización interna).

Lo que se hace es que el servidor envíe una señal a los clientes para que le envíen su hora. Basándose en el RTT de las conversaciones aproxima el tiempo de ida del primer mensaje a cada cliente. A partir de esto calcula un promedio de los relojes (incluyéndose) para que todos se ajusten a esa hora. Entonces lo que hace es enviar a cada cliente la diferencia que tienen que ajustar a su reloj para estar coordinados, haciendo que esta fase no añada error en la sincronización.

10-Pasos a realizar para ejecutar:

El ejercicio se realizó utilizando el lenguaje de programación Python, por lo que se debe contar con Python instalado. Luego se ejecuta:

-En máquina de servidor: proyecto1/ejercicio10\$ python server.py

-En máquina cliente: proyecto1/ejercicio10\$ python [client.py](#) <ip servidor>